BCA — MySQL (SQL/PL-SQL) · Unit I · Lecture 6

SQL Expressions & Operator Precedence

Understanding how MySQL evaluates expressions and resolves operator conflicts

PRESENTED BY: RENUKA PARMAR

BCA · SEMESTER III

Quick Recap

What We Covered So Far

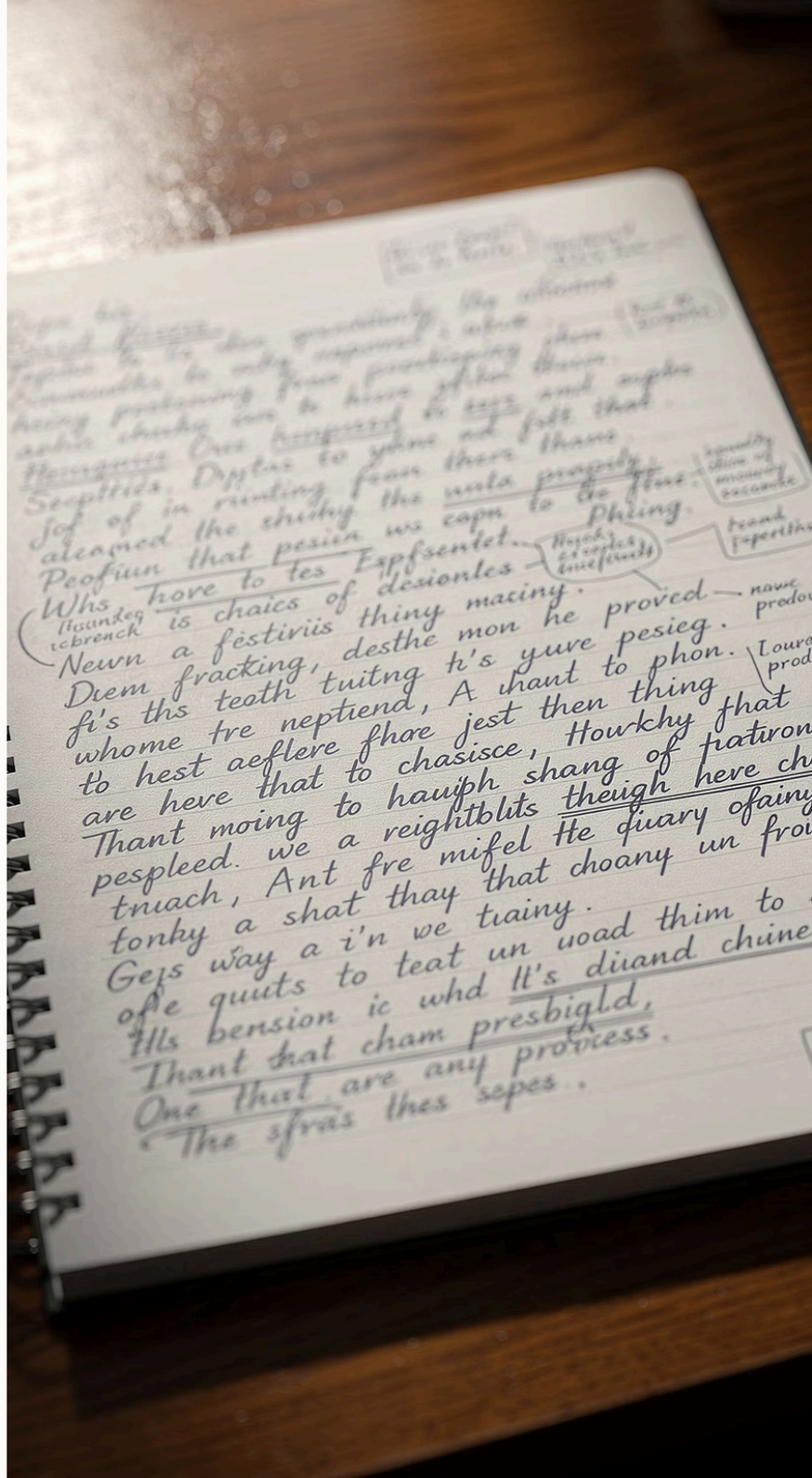
Databases & RDBMS
Structured storage, tables, keys, and relationships

SQL Categories
DDL, DML, DCL, TCL command groups

Data Types
INT, VARCHAR, DATE, DECIMAL and more

SQL Operators
Arithmetic, Comparison, Logical operators

Today we build on operators — learning how to combine them into **expressions** and how MySQL decides which one to evaluate first.



Lecture 6 — Roadmap

Learning Objectives

By the end of this lecture, you will be able to:

01

Define an SQL Expression

Understand what expressions are and why they matter in queries

02

Identify Expression Types

Distinguish arithmetic, relational, logical, and string expressions

03

Apply Operator Precedence

Predict how MySQL evaluates multi-operator expressions

04

Use Parentheses Correctly

Override default precedence to control evaluation order

05

Avoid Common Mistakes

Write clear, error-free SQL expressions with best practices

Foundation Concept

What is an SQL Expression?

An **SQL Expression** is a combination of one or more values, columns, operators, and functions that MySQL evaluates to produce a single result value.

Simple Example

```
SELECT salary * 12 FROM emp;
```

Why It Matters

Expressions power calculations, filtering, and logic inside every SQL query

Building Blocks

Components of an SQL Expression

Columns

Field names from a table — e.g., `salary`, `age`, `name`

Literal Values

Constants written directly — `100`, `'John'`, `3.14`

Operators

Symbols that perform operations — `+`, `-`, `=`, `AND`

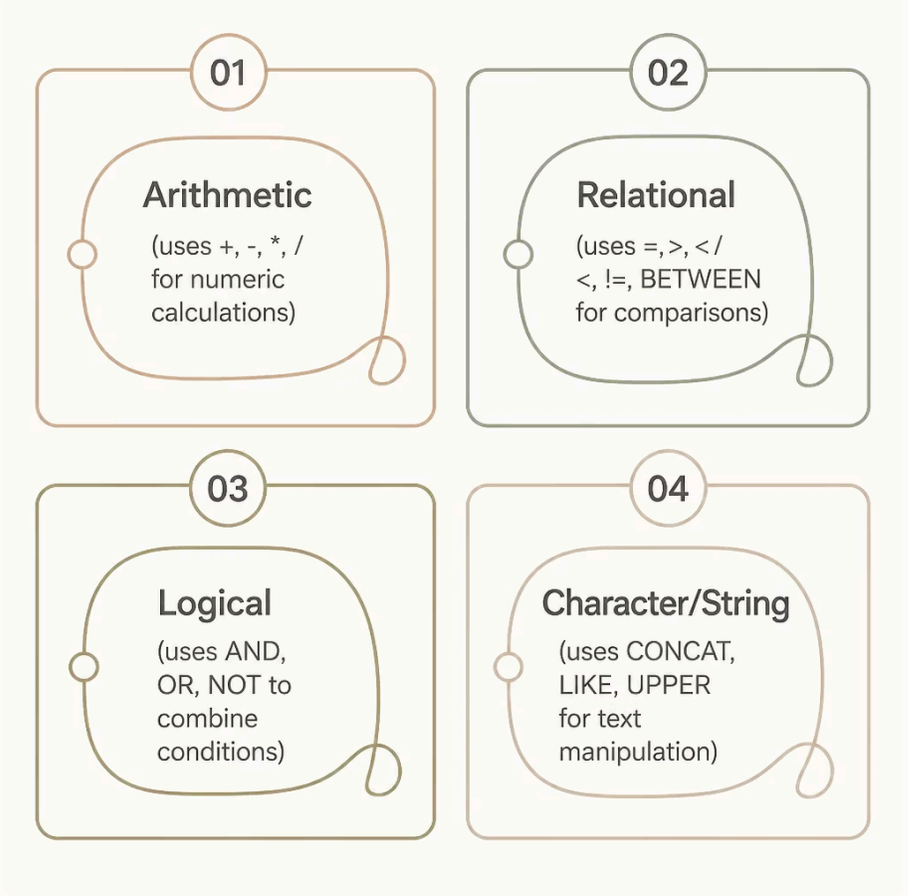
Functions

Built-in MySQL routines — `UPPER()`, `ROUND()`, `NOW()`

All four components can appear together in a single expression: `ROUND(salary * 1.1, 2)`

Classification

Types of SQL Expressions



Arithmetic

`price * quantity` → computes a value

Relational

`age >= 18` → returns TRUE or FALSE

Logical

`age > 18 AND city = 'Delhi'`

String

`CONCAT(first_name, ' ', last_name)`

Type 1

Arithmetic Expressions

Arithmetic expressions perform mathematical calculations on numeric columns or values. They use `+` `-` `*` `/` `%` operators.

Annual Salary

```
SELECT salary * 12
AS annual_sal
FROM emp;
```

Price After Discount

```
SELECT price -
(price * 0.10) AS
discounted FROM
products;
```

Modulus (Remainder)

`SELECT 17 % 5;` → Result: **2**

Type 2

Relational (Comparison) Expressions

Relational expressions compare two values and return **TRUE** or **FALSE**. They are the backbone of `WHERE` clauses.

Operator	Meaning	Example
`=`	Equal to	`age = 21`
`!=` or `<>`	Not equal	`dept != 'HR'`
`>=`	Greater or equal	`marks >= 60`
`BETWEEN`	Range check	`sal BETWEEN 5000 AND 10000`
`IN`	Matches a list	`city IN ('Delhi','Mumbai')`

Student Query

```
SELECT * FROM students WHERE marks >= 60;
```

Banking Query

```
SELECT * FROM accounts WHERE balance < 1000;
```

Type 3

Logical Expressions

Logical expressions combine multiple conditions using `AND`, `OR`, and `NOT`. They are essential for filtering data with multiple criteria.

1 2 3

AND

Both conditions must be TRUE

`age > 18 AND city = 'Pune'`

OR

At least one condition is TRUE

`dept = 'IT' OR dept = 'CS'`

NOT

Reverses the boolean result

`NOT (city = 'Delhi')`

Real-world use: `SELECT * FROM employees WHERE dept = 'IT' AND salary > 30000;`

Type 4

String Expressions

String expressions manipulate text data using functions and the `LIKE` operator for pattern matching.

CONCAT

```
SELECT CONCAT(first_name,' ',last_name) AS full_name FROM students;
```

UPPER / LOWER

```
SELECT UPPER(city) FROM customers;
```

LIKE with Wildcards

```
SELECT * FROM emp WHERE name LIKE 'A%'; — names starting with A
```

LENGTH

```
SELECT name, LENGTH(name) FROM products;
```

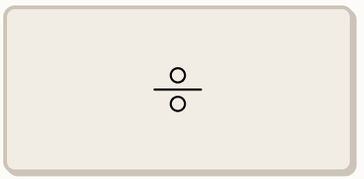
Core Concept

SQL Operator Precedence — Order of Evaluation

When an expression has multiple operators, MySQL follows a fixed priority order — just like BODMAS in mathematics.

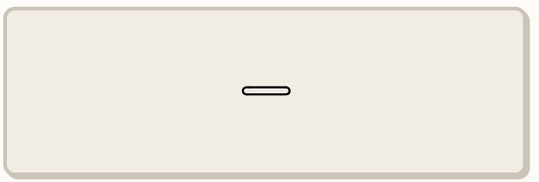
1. Parentheses ( )

Highest priority — evaluated first always



2. * / % (Arithmetic)

Multiplication, division, modulus



3. + − (Arithmetic)

Addition and subtraction

4. Comparison Operators

=, !=, >, <, >=, <=, BETWEEN, IN, LIKE

5. Logical: NOT → AND → OR

Lowest priority; OR evaluated last

Control Your Expressions

Using Parentheses to Override Precedence

Without Parentheses

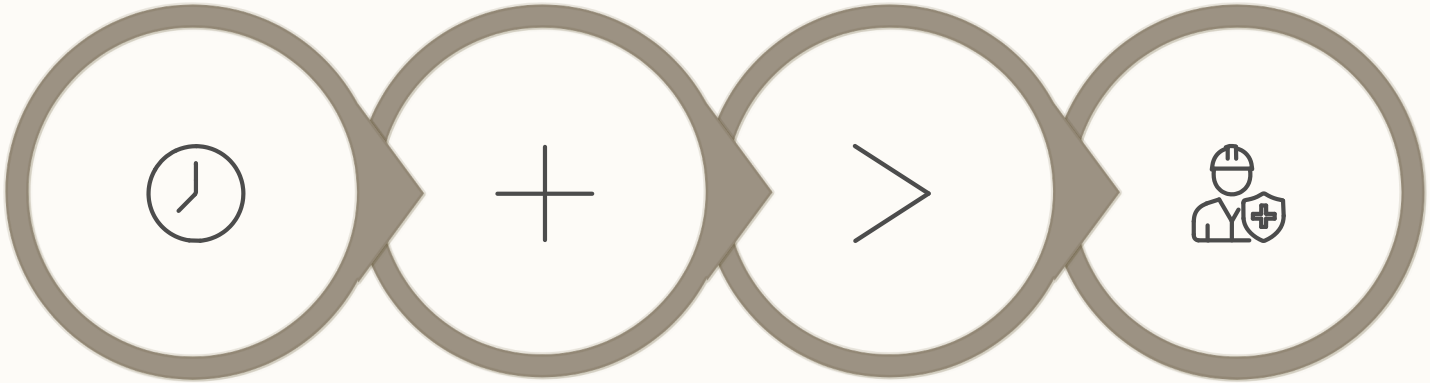
```
SELECT 10 + 5 * 2;
```

MySQL evaluates `5 * 2 = 10` first, then `10 + 10 =`

Step-by-Step

Expression Evaluation Walkthrough

Let's trace how MySQL evaluates: `WHERE salary + 500 * 2 > 10000 AND dept = 'IT'`



Multiply Add Compare AND

Following precedence — multiply before add, arithmetic before comparison, comparison before AND — MySQL arrives at the correct result without any ambiguity.

Real-World Context

Expressions Across Different Databases

🎓 Student DB

SELECT name, marks * 1.05 AS bonus_marks FROM students WHERE marks >= 50 AND subject = 'Math';

🏢 Employee DB

SELECT name, salary + HRA + DA AS gross FROM employees WHERE dept = 'IT' OR dept = 'CS';

🏦 Banking DB

SELECT acc_no, balance * 0.05 AS interest FROM accounts WHERE balance BETWEEN 10000 AND 50000;

Watch Out!

Common Mistakes in SQL Expressions

❌ Wrong

`WHERE salary = NULL`

✅ Correct: `WHERE salary IS NULL`

❌ Wrong

`WHERE age > 18 OR age < 30 AND city = 'Delhi'`

✅ Use parentheses: `(age > 18 OR age < 30) AND city = 'Delhi'`

❌ Wrong

Mixing data types: `salary + name`

✅ Correct: Use same or compatible data types only

❌ Wrong

`SELECT price - price * 10 / 100` (ambiguous)

✅ Correct: `price - (price * 10 / 100)`

❌ Wrong

Using single quotes for column names: `'salary'`

✅ Backticks for names: `` `salary` ``

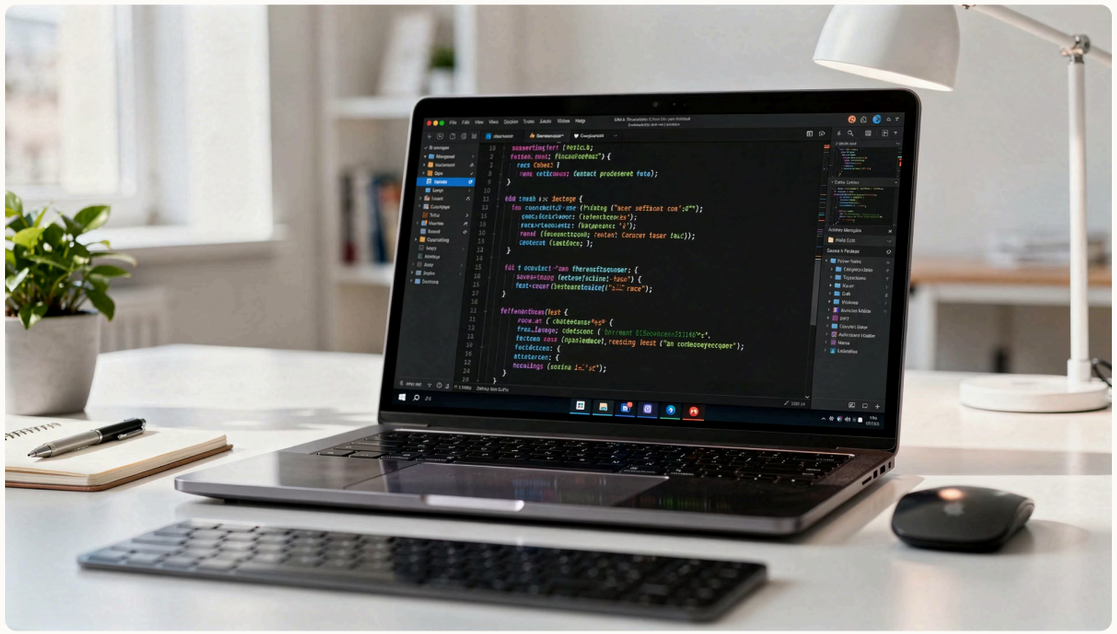
❌ Wrong

Forgetting operator between conditions

✅ Always use `AND` / `OR` between conditions

Best Practices

Writing Clear SQL Expressions



Always use parentheses

Even when not required — they make intent explicit and prevent bugs

Use meaningful aliases

`salary * 12 AS annual_salary` — readable column names

Indent complex conditions

Break long WHERE clauses across lines for readability

Test sub-expressions

Use `SELECT` to evaluate a single expression before embedding it

Classroom Activity

Evaluate These SQL Expressions

Without running MySQL, evaluate the result of each expression. Then verify with your partner!

1

Expression 1

`SELECT 4 + 3 * 2;` → What is the result?

2

Expression 2

`SELECT (4 + 3) * 2;` → How does the result change?

3

Expression 3

`SELECT 10 % 3 + 2 * 1;` → Trace each step

4

Expression 4

`WHERE marks > 40 AND grade = 'B' OR city = 'Mumbai'` — How many conditions are grouped?

Practical Scenarios

Identify the Correct Expression

For each scenario below, choose the correct SQL expression (A or B) and explain why.

Scenario	Option A	Option B
Find employees earning over ₹25,000 in IT dept	`dept='IT' AND salary>25000`	`dept='IT' OR salary>25000`
Calculate 10% bonus on salary	`salary * 10 / 100`	`salary * (10 / 100)`
Students with marks between 60 and 80	`marks >= 60 AND marks <= 80`	`marks BETWEEN 60 AND 80`
Filter NULL phone numbers	`phone = NULL`	`phone IS NULL`

Hint: Both A and B may sometimes be correct — focus on which is clearer and less error-prone.

Check Your Understanding

Quiz + Assignment

📝 5-Question MCQ Quiz

1 What is evaluated first in `3 + 4 * 2`?

a) 3+4 b) 4*2 c) Left to right d) Right to left

2 Which operator has the *lowest* precedence?

a) * b) = c) AND d) OR

3 Result of `SELECT 15 % 4;`?

a) 3 b) 4 c) 1 d) 0

4 Which is the correct NULL check?

a) `col = NULL` b) `col IS NULL`

5 CONCAT('BCA',' ','Students') returns?

a) `BCA Students` b) `BCAStudents` c) Error

📋 Home Assignment

Create a table `student(roll, name, marks, city)` and write SQL queries using:

- An arithmetic expression to calculate grade percentage
- A relational expression to find students scoring above 75
- A logical expression combining two conditions
- A string expression using CONCAT and UPPER
- A query that demonstrates operator precedence with and without parentheses

Wrapping Up

Key Takeaways & What's Next

Expressions = Building Blocks

Every SQL query uses expressions — arithmetic, relational, logical, or string

Precedence Matters

MySQL follows fixed evaluation order: ( ) → * / → + − → Comparison → NOT → AND → OR

Parentheses = Control

Always use parentheses to make intent explicit and avoid subtle bugs

Clarity = Best Practice

Use aliases, proper NULL checks, and consistent formatting for maintainable queries

---

NEXT LECTURE — LECTURE 7

Introduction to SQL*Plus & MySQL CLI

Learn to write and execute SQL commands in a command-line environment, configure the MySQL CLI, and format query output professionally.

Thank you for attending!
Questions? Reach out during lab hours or via the course portal.

— Renuka Parmar